

CSL302: Compiler Design
End Sem Examination (2025-26-M Semester)

Max. Points: 100

Duration: 3 hours

November 28, 2025

1. All questions are compulsory
2. Make your assumptions and state them wherever necessary

Question-1 (Semantic Analysis):

[15 Points]

Part-(a)

Consider the following grammar that generates expressions for a number of operators. Assume you have synthesized attributes **E.min** and **E.max** which should be set to the minimum and maximum values for E, and synthesized attribute **CONST.val** which is the value of the constant. Write the semantic rules that calculate the range (minimum and maximum values) of each subexpression in the respective attributes.

E	→	CONST	{ E.min = ?? ; E.max = ?? }
		ID	{ E.min = ID.min ; E.max = ID.max }
		E1 + E2	{ E.min = ?? ; E.max = ?? }
		E1 - E2	{ E.min = ?? ; E.max = ?? }
		E1 * E2	{ E.min = ?? ; E.max = ?? }
		-E1	{ E.min = ?? ; E.max = ?? }

Part-(b):

Let synthesized attribute F.val give the value of the binary fraction generated by F in the grammar that follows:

$F \rightarrow .L$
 $L \rightarrow LB \mid B$
 $B \rightarrow 0 \mid 1$

For instance, on input .101 we have that F.val = .625

- (a) Using only synthesized attributes, write the semantic rules to compute F.val for the above grammar.
- (b) Show the translation of the input string .101 by annotating its parse tree

Question-2 (Syntax Analysis):

Consider the following grammar.

$$\begin{aligned} S &\rightarrow B \\ B &\rightarrow BE+ \\ &\quad | -E \\ E &\rightarrow e \\ &\quad | Ee \\ &\quad | \epsilon \end{aligned}$$

- (a) Is the grammar LR(1)? Justify your answer by showing DFA and the LR(1) parser table.
- (b) Is the grammar LALR? Justify your answer by reconstructing DFA and the corresponding LALR parser table.

Question-3 (Intermediate Code Generation):

[20 Points]

Assume that your programming language supports control flow instructions for the "For Loop" using the following grammar

foreach-statement $\rightarrow$ foreach(ID in [START..END]) S ;

Semantics are as follows:

1. ID gets initialised to constant values START and executes S iteratively as long as the $ID \leq END$.
2. At the end of each iteration ID gets incremented by 1.

Part-(a): Write down the semantic rules to generate the three address codes as intermediate representation.

Hints: (1) You can use the backpatch function discussed in the class. (2) You can introduce the markers (M, N, etc) into the above grammar with suitable epsilon transitions to pick up index of next quadruple

Part-(b): Generate the three address codes for the following statement with the above semantics.

```
foreach(i in(1..20))
    j=j*b+d;
```

Question-4 (Code Generation):

[30 Points]

Assume that an architecture supports the ternary operations $\oplus$, $\odot$, $\ominus$. Each of which accepts 3 operators.

Example usage of the operators is: $OP(a,b,c)$, where OP can be $\oplus$, $\odot$, or $\ominus$. These operators perform some computation based on the values provided by a , b , and c .

Assume that we have the following target machine model.

Instruction	Semantics
Store m, r	Stores the contents of register r to the memory location m
Load r, m	Loads the contents of the memory location m to the register r
OP $r2, r2, m, r1$	Performs the ternary operation of register $r2$, memory operand m and register operand $r1$. The result of $OP\ r2, m, r1$ is stored in $r2$. Here OP can be $\oplus$, $\odot$, or $\ominus$
OP $r3, r3, r2, r1$	Performs the ternary operation of register $r3$, register $r2$, and register $r1$. The result of $OP\ r3, r2, r1$ is stored in $r3$. Here OP can be $\oplus$, $\odot$, or $\ominus$

- Modify the labelling algorithm that we discussed in the class to calculate the minimum number of registers required to compute an expression tree supporting the above target machine model.
- Modify the Sethi-Ullman Algorithm to generate the code. Discuss various cases of your algorithm for code generation. You can assume that your target has a sufficient number of registers, so you don't need to consider the case of moving to temporary locations.
- Use the modified labelled algorithm and the code generation algorithm to generate the code for the following expression

$$\odot(\oplus(a,b,c),d,\oplus(\odot(e,f,g),\ominus(h,i,j),k))$$

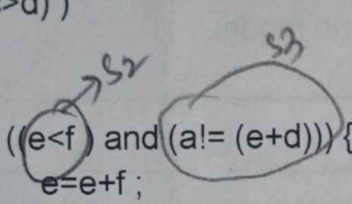
You must draw the expression tree and list the label for each node in the tree, and the final sequence of assembly instructions that are generated for the expression.

[15 Points]

Question-5 (Machine Independent Optimizations):

Consider the following snippet of the code.

```
a=1;
b= a+b;
e= c+d;
if((a<b) or (c>d) )
{
    j=4;
    while ((e<f) and (a!= (e+d))) {
        e=e+f;
        a=c+d;
    }
}
else
{
    b=c+d;
}
```



- Generate the three-address code representation for the above program. Assume that the implementation uses short circuit evaluation for Boolean operators. You can assume the precedence of the associativity follows that of C language.
- Apply the loop invariant code motion and global common sub expression elimination to the above three address codes.
- Apply the copy-propagation for the code generated for the code generated in Part(b) and show the optimized code.

CSL302: Compiler Design
Mid Sem Examination (2025-26-M Semester)

Duration: 2 hours

Max. Points: 100

September 19, 2025

Instructions:

1. All the questions are compulsory
2. Don't ask any unnecessary questions during the exam. If you make any assumptions, write them down clearly.

Question-1:

[20 Points]

Write a lexical analyzer that takes an input file, cleans the input file as per the following rules, and produces an output file which is cleaned.

- a. Replaces all the tab spaces, more than one contiguous space into a single space.
- b. Converts the amount listed in different foreign currencies (\$, €) to Indian currency (₹). You can use the following conversion.
 - i. \$1 = ₹90
 - ii. €1 = ₹100
- c. Deletes unknown values having NULL, UNKNOWN, NOT AVAILABLE, and NA in the input file
- d. Make all the following date formats to the consistent format DD-MM-YYYY
 - i. YYYY-DD-MM to DD-MM-YYYY
 - ii. DD/MM/YYYY to DD-MM-YYYY
- e. Convert the words containing all the capital letters to smaller letters, except the following words.
NASA, ISRO, AI, ML, SQL, CPU, GPU, DBMS, ID

You need not write the complete lex code, but you should write the code corresponding to the rules sections along with any functions you may be requiring.

Question-2:

[7 Points]

1. Show that the following grammar is ambiguous with an example string with two left most or two right most derivations.
$$S \rightarrow S S \mid (S) \mid \epsilon$$
2. Rewrite the above grammar to make it unambiguous. Take the same example as in the first part, and show a unique left-most or right-most derivation.

[20 Points]

Question-3:

Write a CFG for a mini assembly language with the following features.

Tokens: ALU_OP, REG, JUMP, JNZ, HLT_OP, LABEL, CONST

1. An assembly program contains zero or more statements.
2. Each statement can be one of the following:
 - a. Arithmetic statement
 - b. Conditional statement
 - c. Halt statement
3. The arithmetic statement starts with ALU_OP followed by 3 operands, which are separated by commas. The first operands should be REG, the second and third operand can be REG or CONST. Examples are
 - a. ALU_OP REG, CONST, REG
 - b. ALU_OP REG, REG, REG
4. Conditional statements start with JUMP or JNZ. If the statement starts with a JUMP then it is followed by LABEL. If it starts with JNZ, it should have two operands REG, LABEL. The following are two valid examples.
 - a. JUMP LABEL
 - b. JNZ REG LABEL
5. Halt statement starts with HLT_OP and does not have any operands.
6. Each statement can be optionally prefixed with LABEL:

Question-4:

[25 Points]

Consider the following grammar:

$A \rightarrow A x B \mid C y$

$B \rightarrow z A y$

$C \rightarrow x A \mid \epsilon$

Handwritten notes:
 $A \rightarrow \epsilon y A$
 $A \rightarrow x B A \mid \epsilon$

where $\{A, B, C\}$ is the set of nonterminal symbols, A is the start symbol, and $\{x, y, z\}$ is the set of terminal symbols.

- (a) Compute the FIRST and FOLLOW sets. Is the above grammar LL(1)? Justify using the parser table.
- (b) If the grammar is not LL(1), rewrite the grammar to make it LL(1)
- (c) Reconstruct the FIRST, FOLLOW sets, and the LL(1) parser table on the revised grammar and show that the grammar is LL(1).

Handwritten notes for FIRST and FOLLOW sets:
 FIRST: $A \rightarrow x, \epsilon$; $A' \rightarrow x, \epsilon$; $B \rightarrow z$; $C \rightarrow x, \epsilon$
 FOLLOW: $A \rightarrow y$; $A' \rightarrow y$; $B \rightarrow y$; $C \rightarrow y$

Question-5:

[28 Points]

Consider the following grammar:

$S \rightarrow A$

$A \rightarrow A+A \mid B++$

$B \rightarrow y$

- (a) Is the grammar LR(0)? Justify your answer by showing DFA and the LR(0) parser table.
- (b) Trace the input string $y+++y++$ using the LR(0) parser table, and check if it can be accepted by the LR(0) algorithm. If not, show the step where the conflict occurs.
- (c) Repeat the above both Part-(a) and Part-(b) for the SLR algorithm.

